

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 03 -

INTRODUÇÃO AO APIS RESTFUL

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	15
REFERÊNCIAS.....	16

EMSE

O QUE VEM POR AÍ?

Nesta aula, exploraremos o importante papel das APIs na comunicação entre aplicações de software. Começaremos com uma introdução a APIs, analisando sua definição e os princípios fundamentais que as sustentam. Você compreenderá o que é uma API, seu significado e como funcionam, proporcionando uma base sólida para entender sua relevância e operação.

Prosseguiremos examinando os diferentes tipos de APIs, como SOAP, RPC, Websocket e REST, destacando suas características e aplicações práticas. Vamos abordar as integrações de API, seus benefícios e como elas facilitam a inovação e a expansão de funcionalidades. Também discutiremos a segurança e a performance dos endpoints de API, com ênfase em como proteger uma API REST utilizando tokens de autenticação e chaves de API. Por fim, analisaremos recursos, métodos HTTP, códigos de status e headers, além das melhores práticas para o design de URI em APIs REST.

HANDS ON

Nesta aula, vamos aprender sobre os conceitos de uma API, fundamental para a comunicação entre aplicações de software. Abordaremos a definição e os princípios básicos das APIs, entendendo seu funcionamento e importância. Analisaremos os diferentes tipos de APIs, como SOAP, RPC, Websocket e REST, destacando suas características e aplicações práticas.

Exploraremos também a segurança e a performance dos endpoints de API, com foco em proteger uma API REST usando tokens de autenticação e chaves de API. Discutiremos recursos, métodos HTTP, códigos de status e headers, além das melhores práticas para o design de URI em APIs REST. Cada tópico será explicado de maneira detalhada, proporcionando uma base sólida de conhecimento teórico que será essencial quando chegarmos à prática de desenvolvimento de APIs.

SAIBA MAIS

APIs REST, ou Representational State Transfer, representam uma abordagem bastante adotada para o desenvolvimento de serviços web, tornando-se essencial para qualquer pessoa desenvolvedora que busca criar aplicações modernas e eficientes. Imagine um universo onde diferentes sistemas – sejam eles aplicações web, móveis, desktop ou dispositivos IoT – podem se comunicar de forma fluida e padronizada. Este é o mundo que as APIs REST nos permitem construir.

A arquitetura REST é fundamental para a comunicação entre sistemas. Ao usar requisições HTTP simples, como GET, POST, PUT e DELETE, ela facilita a interação entre diferentes componentes de um sistema, independentemente da linguagem de programação ou plataforma em que foram desenvolvidos. Isso não só promove a interoperabilidade, mas também garante uma comunicação eficiente e consistente.

Um dos grandes benefícios das APIs REST é a escalabilidade. Em um ambiente RESTful, há uma separação clara entre cliente e servidor, permitindo que ambos possam ser escalados de forma independente. Isso significa que, conforme sua aplicação cresce, você pode aumentar a capacidade de processamento ou armazenamento do servidor sem precisar alterar o cliente e vice-versa. Essa modularidade também torna a manutenção e a atualização do sistema muito mais gerenciáveis, uma vez que cada parte pode ser modificada sem afetar o todo.

A simplicidade é outro ponto forte das APIs REST. O uso de protocolos HTTP, algo com que a maioria das pessoas desenvolvedoras já está familiarizada, reduz a curva de aprendizado e facilita a implementação. Não é necessário entender complexidades excessivas para começar a trabalhar com REST. A simplicidade do protocolo torna o processo de desenvolvimento mais rápido e menos propenso a erros.

Quando falamos em desempenho, REST também se destaca. A arquitetura permite a implementação de técnicas de caching, o uso de URLs amigáveis e um gerenciamento eficiente de sessões, tudo contribuindo para uma experiência de usuário mais rápida e responsiva. Isso é crucial em um mundo onde a velocidade e a eficiência são essenciais para manter os usuários satisfeitos.

A flexibilidade e a extensibilidade são inerentes ao design RESTful. Não há restrições severas sobre o formato dos dados ou como os endpoints devem ser organizados. Isso permite que a API se adapte às necessidades específicas do seu projeto. A arquitetura baseada em recursos, onde cada recurso é identificado por um URL único, promove um design claro e organizado que reflete diretamente os dados e operações do domínio da aplicação.

Além disso, segurança é uma preocupação constante no desenvolvimento de qualquer aplicação e as APIs REST abordam isso de várias maneiras. Mecanismos como OAuth, tokens de autenticação, HTTPS e controles de acesso baseados em roles garantem que apenas usuários autorizados possam acessar e modificar os dados. Isso protege tanto a integridade quanto a confidencialidade das informações.

A documentação é outro aspecto vital das APIs REST. Uma API bem documentada permite que pessoas desenvolvedoras entendam rapidamente como interagir com os serviços oferecidos. Ferramentas como Swagger e OpenAPI tornam mais fácil criar e manter essa documentação, além de possibilitar a geração automática de clientes e servidores. Isso não só economiza tempo, mas também reduz a possibilidade de erros de implementação.

A aplicabilidade das APIs REST é vasta. Elas são usadas para fornecer dados a aplicações web e móveis, suportando funcionalidades como autenticação e operações CRUD (Create, Read, Update, Delete). Em arquiteturas de microsserviços, cada serviço pode expor uma API REST, permitindo uma comunicação desacoplada entre diferentes componentes da aplicação. Empresas frequentemente utilizam APIs REST para integrar sistemas internos ou oferecer serviços a terceiros, como provedores de pagamento, serviços de geolocalização e redes sociais.

Ao mergulharmos no universo das APIs REST, é fundamental compreendermos a base sobre a qual elas operam: o protocolo HTTP (Hypertext Transfer Protocol). HTTP é a espinha dorsal das comunicações na web e entender seu funcionamento é crucial para desenvolver e utilizar APIs REST de forma eficaz.

O HTTP é um protocolo de aplicação, projetado inicialmente para a comunicação entre navegadores web e servidores. No entanto, sua simplicidade e eficácia o tornaram a escolha ideal para uma ampla gama de aplicações de rede. O

protocolo opera no modelo cliente-servidor, onde o cliente envia requisições e o servidor responde, proporcionando um ciclo de interação claro e organizado.

APIs REST aproveitam os métodos HTTP para realizar operações sobre recursos. Os métodos mais comuns são:

- **GET**: usado para recuperar informações de um servidor. Em uma API REST, um GET request a um endpoint específico retorna a representação de um recurso.
- **POST**: utilizado para criar novos recursos no servidor. Enviando dados no corpo da requisição, o servidor processa e cria um novo registro.
- **PUT**: empregado para atualizar recursos existentes. Similar ao POST, mas direcionado a um recurso específico identificado pelo URL.
- **DELETE**: utilizado para remover recursos do servidor, permitindo uma gestão completa do ciclo de vida do recurso através de uma API REST.

No contexto das APIs REST, cada recurso é identificado por um URL único, que atua como um endereço para acessar ou manipular o recurso. Essa abordagem clara e intuitiva facilita a navegação e a interação com os dados. Por exemplo, em uma API que gerencia uma biblioteca, o recurso para um livro específico poderia ser acessado através de um URL como “http://api.biblioteca.com/livros/123”.

O HTTP também fornece uma série de códigos de status que são retornados pelo servidor para indicar o resultado da requisição. Esses códigos são vitais para a comunicação clara entre cliente e servidor.

Já os cabeçalhos HTTP desempenham um papel crucial na passagem de informações adicionais com as requisições e respostas. Eles podem incluir informações sobre autenticação (como tokens de autorização), o formato de dados esperado (por exemplo, Content-Type: application/json), e políticas de cache, entre outros. Existem inúmeros cabeçalhos https para diferentes tipos de situações:

Cabeçalhos de autenticação

- **WWW-Authenticate**: método de autenticação necessário para acessar o recurso.
- **Authorization**: credenciais para autenticar um usuário com o servidor.

- Proxy-Authenticate: método de autenticação necessário para acessar recursos através de um proxy.
- Proxy-Authorization: credenciais para autenticar um usuário com o proxy.

Cabeçalhos de cache

- Age: tempo em segundos que o objeto esteve em um cache de proxy.
- Cache-Control: diretrizes para mecanismos de cache.
- Expires: data/hora em que a resposta se torna obsoleta.
- Pragma: usado para compatibilidade com caches HTTP/1.0.
- Warning: informações sobre possíveis problemas.

Cabeçalhos condicionais

- Last-Modified: data da última modificação do recurso.
- ETag: identificador único da versão do recurso.
- If-Match: requisição condicional com base na ETag.
- If-None-Match: requisição condicional para atualizar caches.
- If-Modified-Since: requisição condicional para transmitir dados apenas se o cache estiver desatualizado.
- If-Unmodified-Since: requisição condicional para garantir coerência de fragmentos.

Cabeçalhos de gerenciamento de conexão

- Connection: controla se uma conexão permanece aberta após a transação.
- Keep-Alive: tempo de permanência da conexão aberta.

Cabeçalhos de negociação de conteúdo

- Accept: tipos de dados que podem ser retornados pelo servidor.
- Accept-Charset: conjuntos de caracteres que o cliente pode entender.

- Accept-Encoding: algoritmos de codificação que podem ser usados.
- Accept-Language: linguagem esperada na resposta.

Cabeçalhos de cookies

- Cookie: contém cookies HTTP enviados pelo servidor.
- Set-Cookie: envia cookies do servidor para o cliente.

Cabeçalhos de CORS

- Access-Control-Allow-Origin: indica se a resposta pode ser compartilhada.
- Access-Control-Allow-Credentials: indica se a resposta pode ser exposta com credenciais.
- Access-Control-Allow-Headers: cabeçalhos permitidos na solicitação real.
- Access-Control-Allow-Methods: métodos permitidos na solicitação real.
- Access-Control-Expose-Headers: cabeçalhos expostos na resposta.
- Access-Control-Max-Age: tempo de cache dos resultados de uma solicitação preflight.
- Access-Control-Request-Headers: cabeçalhos usados na solicitação real.
- Access-Control-Request-Method: método usado na solicitação real.
- Origin: indica a origem da requisição.

Cabeçalhos de segurança

- Content-Security-Policy (CSP): controla os recursos que o agente do usuário pode carregar.
- Strict-Transport-Security (HSTS): força o uso de HTTPS.
- X-Content-Type-Options: desativa a análise MIME.
- X-Frame-Options (XFO): indica se uma página pode ser exibida em um <frame>.

- X-XSS-Protection: habilita a filtragem contra cross-site scripting.

Outro tópico importante, abordado nas aulas, utilizado na comunicação http é o status code. Também existem diversos status:

Respostas informativas (100 – 199)

Essas respostas são provisórias e indicam que o servidor está processando a solicitação:

- 100 Continue: o cliente deve continuar com a solicitação.
- 101 Switching Protocols: o servidor está mudando para o protocolo solicitado pelo cabeçalho de solicitação Upgrade do cliente.
- 102 Processing (WebDAV): o servidor recebeu e está processando a solicitação, mas ainda não há resposta disponível.
- 103 Early Hints: permite que o agente do usuário inicie o pré-carregamento de recursos enquanto o servidor prepara uma resposta.

Respostas bem-sucedidas (200 – 299)

Estas respostas indicam que a solicitação foi bem-sucedida:

- 200 OK: a solicitação foi bem-sucedida. O conteúdo do sucesso varia conforme o método HTTP usado (GET, POST, etc.).
- 201 Created: a solicitação foi bem-sucedida e um novo recurso foi criado.
- 202 Accepted: a solicitação foi recebida, mas ainda não foi processada.
- 203 Non-Authoritative Information: os metadados retornados não são exatamente os mesmos que estão disponíveis no servidor de origem.
- 204 No Content: não há conteúdo para enviar para esta solicitação.
- 205 Reset Content: o agente do usuário deve redefinir o documento que enviou esta solicitação.
- 206 Partial Content: usado quando o cabeçalho Range é enviado do cliente para solicitar apenas parte de um recurso.

- 207 Multi-Status (WebDAV): transmite informações sobre vários recursos.
- 208 Already Reported (WebDAV): evita a repetição de informações em uma resposta.
- 226 IM Used (HTTP Delta encoding): a resposta é uma representação do resultado de uma ou mais manipulações de instância aplicadas à instância atual.

Mensagens de redirecionamento (300 – 399)

Estas respostas indicam que o cliente precisa tomar mais medidas para concluir a solicitação:

- 300 Multiple Choices: a solicitação tem mais de uma resposta possível.
- 301 Moved Permanently: a URL do recurso solicitado foi alterada permanentemente.
- 302 Found: a URL do recurso solicitado foi alterada temporariamente.
- 303 See Other: direciona o cliente a obter o recurso solicitado em outro URI com uma solicitação GET.
- 304 Not Modified: usado para fins de cache; o cliente pode continuar a usar a mesma versão em cache da resposta.
- 305 Use Proxy (Deprecated): a solicitação deve ser acessada por um proxy (descontinuado por questões de segurança).
- 306 unused (Deprecated): não é mais usado, apenas reservado.
- 307 Temporary Redirect: direciona o cliente a obter o recurso solicitado em outra URI com o mesmo método.
- 308 Permanent Redirect: o recurso está permanentemente localizado em outro URI, especificado pelo cabeçalho Location.

Respostas de erro do cliente (400 – 499)

Estas respostas indicam que houve um erro na solicitação do cliente:

- 400 Bad Request: o servidor não pode processar a solicitação devido a um erro do cliente.
- 401 Unauthorized: o cliente deve se autenticar para obter a resposta solicitada.
- 402 Payment Required: reservado para uso futuro, raramente usado.
- 403 Forbidden: o cliente não tem permissão para acessar o conteúdo.
- 404 Not Found: o servidor não pode encontrar o recurso solicitado.
- 405 Method Not Allowed: o método da solicitação não é suportado pelo recurso de destino.
- 406 Not Acceptable: o servidor não encontra nenhum conteúdo que esteja em conformidade com os critérios fornecidos pelo agente do usuário.
- 407 Proxy Authentication Required: a autenticação precisa ser feita por um proxy.
- 408 Request Timeout: o servidor gostaria de desligar a conexão ociosa.
- 409 Conflict: a solicitação conflita com o estado atual do servidor.
- 410 Gone: o conteúdo solicitado foi excluído permanentemente sem endereço de encaminhamento.
- 411 Length Required: o campo de cabeçalho Content-Length não está definido e é exigido pelo servidor.
- 412 Precondition Failed: o servidor não atende a pré-condições especificadas pelo cliente.
- 413 Payload Too Large: a entidade de solicitação é maior do que os limites definidos pelo servidor.
- 414 URI Too Long: o URI solicitado é mais longo do que o servidor está disposto a interpretar.
- 415 Unsupported Media Type: o formato de mídia dos dados requisitados não é suportado pelo servidor.

- 416 Range Not Satisfiable: o intervalo especificado não pode ser atendido.
- 417 Expectation Failed: a expectativa indicada pelo campo de cabeçalho Expect não pode ser atendida.
- 418 I'm a teapot: o servidor recusa a tentativa de coar café em um bule de chá.
- 421 Misdirected Request: a requisição foi direcionada a um servidor inapto a produzir a resposta.
- 422 Unprocessable Content (WebDAV): a solicitação foi bem formada, mas não foi atendida devido a erros semânticos.
- 423 Locked (WebDAV): o recurso está bloqueado.
- 424 Failed Dependency (WebDAV): a solicitação falhou devido à falha de uma solicitação anterior.
- 425 Too Early: o servidor não está disposto a processar uma solicitação que pode ser repetida.
- 426 Upgrade Required: o servidor se recusa a executar a solicitação usando o protocolo atual.
- 428 Precondition Required: a solicitação deve ser condicional para prevenir conflitos.
- 429 Too Many Requests: o usuário enviou muitas solicitações em um dado tempo.
- 431 Request Header Fields Too Large: os campos de cabeçalho da solicitação são muito grandes.
- 451 Unavailable For Legal Reasons: o recurso solicitado não pode ser fornecido legalmente.

Respostas de erro do servidor (500 – 599)

Estas respostas indicam que houve um erro no servidor:

- 500 Internal Server Error: o servidor encontrou uma situação com a qual não sabe lidar.
- 501 Not Implemented: o método da requisição não é suportado pelo servidor.
- 502 Bad Gateway: o servidor obteve uma resposta inválida ao trabalhar como um gateway.
- 503 Service Unavailable: o servidor não está pronto para manipular a solicitação (manutenção ou sobrecarga).
- 504 Gateway Timeout: o servidor, atuando como um gateway, não conseguiu obter uma resposta a tempo.
- 505 HTTP Version Not Supported: a versão HTTP usada na solicitação não é suportada pelo servidor.
- 506 Variant Also Negotiates: erro de configuração interna do servidor.
- 507 Insufficient Storage (WebDAV): o servidor não pode armazenar a representação necessária para concluir a solicitação.
- 508 Loop Detected (WebDAV): loop infinito detectado ao processar a solicitação.
- 510 Not Extended: extensões adicionais são necessárias para atender à solicitação.
- 511 Network Authentication Required: o cliente precisa se autenticar para obter acesso à rede.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a definição e os princípios fundamentais de APIs RESTful, recursos, métodos HTTP, códigos de status, headers e boas práticas para design de URI. Além disso, estudamos o uso dos métodos GET, POST, PUT, DELETE e a comunicação de diferentes estados da API através dos códigos de status.

Primeiro, aprendemos sobre a definição e os princípios fundamentais das APIs RESTful. Esses princípios nos ajudaram a entender a base teórica por trás do design de APIs RESTful.

Em seguida, abordamos os recursos, métodos HTTP, códigos de status e headers utilizados nas APIs RESTful. Focamos também nas boas práticas para o design de URIs. Essas práticas nos ajudaram a criar URIs claras e intuitivas para nossa API. Além disso, analisamos em detalhes quando e como utilizar os métodos GET, POST, PUT e DELETE. Essa compreensão nos permitiu aplicar corretamente os métodos HTTP em nossas operações de CRUD.

Por fim, discutimos a comunicação de diferentes estados da API utilizando códigos de status HTTP apropriados. Aprendemos a importância de utilizar códigos para indicar o resultado das operações e fornecer feedback claro aos clientes da API.

REFERÊNCIAS

AWS. **O que é uma API (Interface de Programação de Aplicações)?**. 2024. Disponível em: <<https://aws.amazon.com/pt/what-is/api/>>. Acesso em: 02 ago. 2024.

MOZILLA. **Cabeçalhos HTTP**. 2024. Disponível em: <<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Headers>>. Acesso em: 02 ago. 2024.

MOZILLA. **Códigos de status de respostas HTTP**. 2024. Disponível em: <<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status>>. Acesso em: 02 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: API. DD. SPRING BOOT. MVC.

EMSE



POSTECH